

数字设计与计算机原理复习要点

第六章：DDCA Chapter 4 – 硬件描述语言 HDL

1 HDL 导论 (pp.3–6)

- **HDL** (Hardware Description Language): 仅指定逻辑功能, CAD 工具综合出优化门电路 [p.3]
- 两大主要 HDL:
 - **SystemVerilog/Verilog**: 1984 年 Gateway Design Automation 开发, IEEE 1364 (1995 年), IEEE 1800 (2005 年扩展)
 - **VHDL**: 1981 年美国国防部开发, IEEE 1076 (1987 年), 2008 年更新
- **仿真** (Simulation): 施加输入, 检查输出, 在仿真中调试可省数百万美元 [p.4]
- **综合** (Synthesis): 将 HDL 代码转换为门级网表 (Netlist) [p.4]
- **核心原则**: 写 HDL 时, 始终想着代码对应的硬件 [p.5]

2 组合逻辑的 HDL 描述 (pp.7–35)

Verilog 基本语法

- **module**: 基本设计单元, 定义输入/输出端口
- **assign**: 连续赋值语句, 用于组合逻辑
- **wire**: 内部连接信号
- **运算符**: & (AND), | (OR), ^ (XOR), ~ (NOT)

位运算符 vs 逻辑运算符

- **位运算**: &, |, ^ (对多位信号逐位操作)
- **归约运算**: &, |, ^ (单目, 将所有位归约到 1 位)
例: `&A = A[3]&A[2]&A[1]&A[0]`
- **逻辑运算**: &&, || (整体真值判断)

条件赋值

- `assign Y = (S) ? D1 : D0;` – 2:1 Mux
- `assign Y = S[1] ? (S[0] ? D3 : D2) : (S[0] ? D1 : D0);` – 4:1 Mux

内部信号与中间节点

`wire n1, n2;` – 声明内部连线, 用于连接子表达式

3 结构建模 (Structural Modeling) (pp.36–45)

- 通过实例化已有模块来构建层次化设计
- `module_name instance_name (.port1(wire1), .port2(wire2));`
- 支持层次化 (Hierarchy): 复杂设计由子模块构成
- 端口连接方式: 按名称 (`.port_name(wire)`) 或按位置

4 时序逻辑的 HDL 描述 (pp.46–60)

always 块与寄存器

- `always @(posedge clk):` 时钟上升沿触发
- `always @(posedge clk, posedge reset):` 异步复位
- `reg:` 在 `always` 块中赋值的变量
- `<=` (非阻塞赋值): 用于时序逻辑
- `=` (阻塞赋值): 用于组合逻辑

D 触发器 Verilog 示例

```
module d_flipflop(input clk, input d, output reg q);
    always @(posedge clk)
        q <= d;
endmodule
```

带复位的 D 触发器

```
module dff_reset(input clk, reset, input d, output reg q);
    always @(posedge clk, posedge reset)
        if (reset) q <= 0;
        else      q <= d;
endmodule
```

5 更多组合逻辑: case 与 always_comb (pp.61–75)

always_comb 块

- 用于复杂组合逻辑, SystemVerilog 特性
- 所有在块内读取的信号自动加入敏感列表
- 避免了传统 `always (*)` 可能的锁存器推断问题

case 语句

```
always_comb
    case (sel)
        2'b00: y = d0;
```

```

    2'b01: y = d1;
    2'b10: y = d2;
    2'b11: y = d3;
    default: y = 1'b0;
endcase

```

避免意外锁存器

- 在组合 `always` 块中，必须为所有分支赋值
- 使用 `default` 覆盖未处理情况
- 或者初始化所有输出为默认值

6 有限状态机的 HDL 实现 (pp.76–90)

- 三段式 FSM 编码模式:
 1. 状态寄存器 (时序 `always` 块)
 2. 次态逻辑 (组合 `always_comb` 块)
 3. 输出逻辑 (组合 `always_comb` 块 – Moore) 或 (Mealy)

7 参数化模块 (Parameterized Modules) (pp.91–95)

```

module mux #(parameter WIDTH = 8)
    (input [WIDTH-1:0] d0, d1, input s, output [WIDTH-1:0] y);
    assign y = s ? d1 : d0;
endmodule

```

- **parameter:** 编译时常量，可在实例化时覆盖
- **实例化:** `mux #(.WIDTH(16)) mux16(...);`

8 测试平台 (Testbenches) (pp.96–105)

- **Testbench:** 用于仿真验证设计的 HDL 模块
- **特点:**
 - 无输入输出端口 (自包含)
 - 实例化待测模块 (DUT – Device Under Test)
 - 施加测试向量 (Test Vectors)
 - 使用 `$display` 输出结果, `assert` 检查正确性

简单 Testbench 示例

```

module testbench;
    reg a, b;
    wire y;
    and_gate dut(.a(a), .b(b), .y(y));

```

```
initial begin
    a=0; b=0; #10;
    a=0; b=1; #10;
    a=1; b=0; #10;
    a=1; b=1; #10;
    \$.finish;
end
endmodule
```

9 HDL 编码技巧与易错点

- 组合逻辑用 `assign` 或 `always_comb`; 时序逻辑用 `always @(posedge clk)`
- 非阻塞赋值 `<=` 用于时序逻辑, 阻塞赋值 `=` 用于组合逻辑
- 组合 `always` 块中避免意外锁存器: 所有分支赋值完整
- **Digital + HDL 混合使用**: 先用 Digital 画图验证, 再导出 Verilog [p.6]
- 写 HDL 时始终思考对应的硬件电路